

Page Cache, Reclaim und Swap

Wie Dateizugriffe und Speicherdruck zusammenwirken

5.04

Drei Fragen strukturieren den Speicherpfad

01 Was wird gecacht?

Folios und `address_space` halten Dateidaten im Arbeitsspeicher.

02 Wann wird geschrieben?

Dirty-Grenzen, Writeback und `fsync()` steuern die Persistenz.

03 Was wird verdrängt?

Reclaim wählt Kandidaten; Swap sichert anonyme Seiten.

Lernziel

Am Ende kannst du Cache, Writeback und Reclaim anhand von Kernelzustand erklären.

Vier Übergänge prägen den Speicherpfad

Nachschlagen

Der Index sucht ein Folio im Page Cache.

Einlesen

Ein Cache Miss löst Dateisystem-I/O und Readahead aus.

Verändern

Schreibzugriffe markieren Folios dirty; Writeback persistiert.

Freigeben

Reclaim verwirft Clean Cache oder lagert anonym aus.

SPEICHERPFAD

Treffer: Dateiindex → XArray → Folio

Änderung: Dirty → Writeback → Clean

Druck: Scan → Reclaim → Wiederverwendung oder Swap

Folio und address_space bilden den Page Cache

KERNSTRUKTUR

```
struct folio
mapping / index
flags / refcount
address_space
i_pages (XArray)
```

Was der Cache verbindet ?

01 Objekt

Ein Folio bündelt physische Seiten und ihren Zustand.

02 Index

Die XArray ordnet Dateioffsets den Folios zu.

03 Ownership

Das Mapping verbindet Folio, Datei oder anonymen Speicher.

Der Read-Pfad sucht zuerst im Page Cache

QUELLPFAD

```
mm/filemap.c  
filemap_read()  
filemap_get_pages()  
filemap_get_folio()  
read_folio()
```

Lesen in drei Schritten

- 01 Nachschlagen**
Dateiindex in der XArray suchen
- 02 Cache Miss**
Folio anlegen und Dateisystem-I/O anstoßen
- 03 Bereitstellen**
Uptodate-Daten in den User Space kopieren

Folio-Flags machen Cachezustand sichtbar

ZUSTÄNDE

Uptodate

Dirty

Writeback

Referenced

Locked

Zwei Sichtweisen

I/O-Sicht

Uptodate, Dirty und Writeback beschreiben Datenfluss

Reclaim-Sicht

Referenzen, Alter und Mapping beeinflussen Verdrängung

Statuswechsel sind synchronisiert; ein einzelnes Flag erklärt nie den ganzen Pfad.

filemap_get_folio() kapselt die Cache-Suche

MM/FILEMAP.C

```
struct folio *folio;  
folio = filemap_get_folio(mapping, index);  
if (IS_ERR(folio)) {  
    /* Cache Miss behandeln */  
}
```

Lies als Lookup

01

Schlüssel

mapping und index adressieren Dateidaten

02

Treffer

Vorhandenes Folio wird referenziert zurückgegeben

03

Fehlerpfad

Fehlender Eintrag führt zum Einlesen oder zur Fehlerbehandlung

Konkrete Helfer und Rückgaberegeln hängen von der Kernelversion ab.

LRU-Listen und Generationen ordnen Kandidaten

RECLAIM-DATEN

```
file-backed / anonymous  
active / inactive  
referenced / refault  
zone / NUMA node  
memory cgroup
```

Drei Konsequenzen

- 01 Lokalität**
Reclaim arbeitet pro Zone, Node und gegebenenfalls memcg.
- 02 Nutzung**
Referenzen und Refaults schützen häufig gebrauchte Daten.
- 03 Balance**
Dateicache und anonyme Seiten konkurrieren um Speicher.

Reclaim kann im Hintergrund oder direkt laufen

RECLAIM-WEGE

```
kswapd  
direct reclaim  
memcg reclaim  
shrink_lruvec()  
shrink_folio_list()
```

Vom Druck zur Freigabe

- 01 Hintergrund**
kswapd reagiert auf niedrige Zonen-Watermarks.
- 02 Direkt**
Eine Allokation scannt selbst, wenn Speicher knapp ist.
- 03 Begrenzt**
Ein Memory Cgroup kann unabhängig unter Druck geraten.

Direkter Reclaim verlängert den kritischen Pfad einer Allokation.

Writeback macht Dirty-Dateidaten persistent

KERNFUNKTIONEN

```
balance_dirty_pages()  
writeback_inodes_wb()  
writepages()  
folio_start_writeback()
```

Zwei Ebenen

Datenfluss

Dirty wird Writeback und nach erfolgreichem I/O wieder Clean.

Flusskontrolle

Dirty-Grenzen drosseln Schreiber und begrenzen Rückstände.

Hintergrund-Writeback ersetzt keine explizite Persistenzgarantie durch `fsync()`.

Scannen, prüfen und freigeben bilden den Reclaim-Kreis

RECLAIM

```
watermark_low
shrink_node()
shrink_lruvec()
folio_check_references()
shrink_folio_list()
```

Der Kreislauf

01 Auslöser

Watermark oder Allokationsfehler erzeugt Speicherdruck.

02 Bewertung

Nutzung, Typ und Zustand entscheiden über den Kandidaten.

03 Freigabe

Clean Cache wird verworfen; Dirty benötigt Writeback.

Gesperrte, gemappte oder kürzlich genutzte Folios sind schwerer zurückzugewinnen.

Swap erweitert Reclaim für anonymen Speicher

MM/SWAP_STATE.C

```
add_to_swap()  
swap_writepage()  
swpin_readahead()  
delete_from_swap_cache()
```

Drei Phasen

- 01 Swap-Slot**
Anonymer Inhalt erhält einen persistenten Ablageort.
- 02 Swap-out**
Daten werden geschrieben; der RAM kann freigegeben werden.
- 03 Swap-in**
Ein Page Fault lädt den Inhalt bei erneutem Zugriff.

Praxis: Cache und Speicherdruck beobachten

TERMINAL

```
$ grep -E 'Cached|Dirty|Writeback|Swap' /proc/meminfo
$ vmstat 1
$ cat /proc/pressure/memory
$ grep -E 'pgscan|pgsteal|pswp' /proc/vmstat
```

Belege drei Aussagen

01

Cache

Vor und nach einem wiederholten Dateizugriff vergleichen

02

Reclaim

Scan, Steal und Pressure unter kontrollierter Last verfolgen

03

Swap

Swap-I/O beobachten oder dessen Abwesenheit begründen

Praxis: Den Speicherpfad belegen

01 Baseline sichern

Meminfo, vmstat und PSI vor dem Versuch dokumentieren.

02 Cache füllen

Eine große Datei zweimal lesen und Unterschiede erklären.

03 Dirty erzeugen

Begrenzt schreiben und Dirty sowie Writeback beobachten.

04 Druck erzeugen

Kontrollierte Speicherlast starten und Abbruchgrenze setzen.

05 Reclaim belegen

pgscan, pgsteal und PSI zeitlich zuordnen.

06 Quellpfad finden

Eine beobachtete Kennzahl bis zur Kernel-Funktion verfolgen.

Arbeite in der Laborumgebung; stoppe bei starker Latenz oder beginnendem OOM.

Page Cache koppelt I/O, Speicherdruck und Latenz.

Formuliere in einem Satz:

Wann wird ein Folio wiederverwendet, zurückgeschrieben oder verworfen?